

# Dashboard basics - part 1

## Project Links

Streamlit app link: [https://ind320project-6dme9afld5jrddxdm8ds6.streamlit.app/page\\_4](https://ind320project-6dme9afld5jrddxdm8ds6.streamlit.app/page_4)

GitHub link: [https://github.com/prakum04/IND320\\_Project](https://github.com/prakum04/IND320_Project)

## AI Usage

I used ChatGPT throughout the assignment when I needed help with errors or did not understand how to do something in Python or Streamlit.

I used it when setting up the project environment and installing the packages I needed. I also used it when setting up and running Streamlit and when I had problems connecting the Streamlit application to the project.

For the data analysis, I used ChatGPT to help understand which plots were suitable for the different columns. I also used it when figuring out how to plot all the columns together, since the variables had different scales.

For the Streamlit application, I used ChatGPT when creating the different pages and navigation. I also used it to understand how to use "LineChartColumn()" for the table, and when implementing the selectbox and slider on the plotting page.

When I got errors, I used ChatGPT to understand what the error meant and how I could fix it. I tested the code in my own environment and changed it when needed to make it work with my project and data.

## Work Log

Work Log Reading the reservoirs.csv file and renaming the Norwegian columns to English was part of the initial work. I then inspected the data and started plotting the different columns to better understand the dataset and the variables it contained. Since the variables had different scales, I used MinMaxScaler from Scikit-Learn to scale the measurement variables between 0 and 1. This made it easier to compare several variables in the same plot. I chose to use the library instead of manually normalizing the data. The data was also grouped by date using the mean across reservoir areas to make the combined plot easier to read and understand.

After working with the data in the Jupyter Notebook, I started building the Streamlit application. The application consists of four pages with navigation through a sidebar. The first page is a home page, while the second page displays the imported reservoir data. I used LineChartColumn to display the data series for the first month, with one row for each CSV column. Non-numeric columns are also included in the table but do not have a line chart.

The third page focuses on plotting the data. I added a selectbox that allows the user to select any single CSV column or display all the measurement columns together. I also added a select slider where the user can select a range of months, with the first month as the default selection. When all measurement variables are selected, they are scaled before being plotted together. During the project, I encountered a `FileNotFoundError` when trying to read the `reservoirs.csv` file because the file path was incorrect. I used AI to help me understand the error and find the correct file path. AI also helped me understand how to use relative paths so the file could be accessed correctly in the project. Finally, I pushed the completed project to GitHub and deployed the application using Streamlit Community Cloud.

## Importing libraries

```
In [65]: import pandas as pd
import matplotlib.pyplot as plt
```

## Reading the CSV file

```
In [66]: df = pd.read_csv("../data/reservoirs.csv")
```

## Rename headers to English and understandable

```
In [67]: #Creating a list with the english column names and setting as the column title
header = ["date_id"
, "area_type"
, "area_number"
, "iso_year"
, "iso_week"
, "fill_level"
, "capacity_TWh"
, "stored_energy_Twh"
, "next_publication_date"
, "fill_level_last_week"
, "change_in_fill_level"]

df.columns = header
```

## Printing the data in a relevant way

```
In [68]: #Getting an overview of the data
df.head()
```

Out[68]:

|  | date_id | area_type | area_number | iso_year | iso_week | fill_level | capacity_TWh | stored |
|--|---------|-----------|-------------|----------|----------|------------|--------------|--------|
|--|---------|-----------|-------------|----------|----------|------------|--------------|--------|

|   |            |    |   |      |    |          |           |  |
|---|------------|----|---|------|----|----------|-----------|--|
| 0 | 1995-09-03 | EL | 4 | 1995 | 35 | 0.944721 | 21.079208 |  |
| 1 | 2020-11-15 | EL | 1 | 2020 | 46 | 0.974361 | 6.003264  |  |
| 2 | 2011-08-28 | EL | 4 | 2011 | 34 | 0.786499 | 21.079208 |  |
| 3 | 1998-07-05 | EL | 3 | 1998 | 27 | 0.793969 | 8.920932  |  |
| 4 | 2011-03-27 | EL | 4 | 2011 | 12 | 0.300820 | 21.079208 |  |



In [69]: *#Checking column names, number of non-null values, data types and number of rows*  
`df.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 14877 entries, 0 to 14876
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   date_id                               14877 non-null  str
1   area_type                             14877 non-null  str
2   area_number                           14877 non-null  int64
3   iso_year                              14877 non-null  int64
4   iso_week                              14877 non-null  int64
5   fill_level                            14877 non-null  float64
6   capacity_TWh                          14877 non-null  float64
7   stored_energy_Twh                     14877 non-null  float64
8   next_publication_date                  14877 non-null  str
9   fill_level_last_week                   14877 non-null  float64
10  change_in_fill_level                   14877 non-null  float64
dtypes: float64(5), int64(3), str(3)
memory usage: 1.7 MB
```

In [70]: *#Gives a summary of numerical columns: mean, standard deviation, outliers, min a*  
`df.describe()`

Out[70]:

|              | area_number  | iso_year     | iso_week     | fill_level   | capacity_TWh | stored_e     |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| <b>count</b> | 14877.000000 | 14877.000000 | 14877.000000 | 14877.000000 | 14877.000000 | 14877.000000 |
| <b>mean</b>  | 2.333333     | 2010.346038  | 26.405929    | 0.618852     | 29.145860    | 14877.000000 |
| <b>std</b>   | 1.490762     | 9.146696     | 15.017910    | 0.214850     | 22.739070    | 14877.000000 |
| <b>min</b>   | 0.000000     | 1995.000000  | 1.000000     | 0.055160     | 6.003264     | 14877.000000 |
| <b>25%</b>   | 1.000000     | 2002.000000  | 13.000000    | 0.456234     | 17.390587    | 14877.000000 |
| <b>50%</b>   | 2.000000     | 2010.000000  | 26.000000    | 0.648049     | 23.237715    | 14877.000000 |
| <b>75%</b>   | 3.000000     | 2018.000000  | 39.000000    | 0.802135     | 34.043590    | 14877.000000 |
| <b>max</b>   | 5.000000     | 2026.000000  | 53.000000    | 1.020072     | 87.437580    | 14877.000000 |



In [71]: *#Missing values*  
`df.duplicated().sum()`

Out[71]: `np.int64(0)`

In [72]: `df.nunique()`

Out[72]:

|                                    |       |
|------------------------------------|-------|
| <code>date_id</code>               | 1653  |
| <code>area_type</code>             | 3     |
| <code>area_number</code>           | 6     |
| <code>iso_year</code>              | 32    |
| <code>iso_week</code>              | 53    |
| <code>fill_level</code>            | 14868 |
| <code>capacity_TWh</code>          | 9     |
| <code>stored_energy_TWh</code>     | 14871 |
| <code>next_publication_date</code> | 396   |
| <code>fill_level_last_week</code>  | 14868 |
| <code>change_in_fill_level</code>  | 14828 |

`dtype: int64`

Plot each column separately

In [73]: *#Illustrating three area types: EL, Vass and NO.*

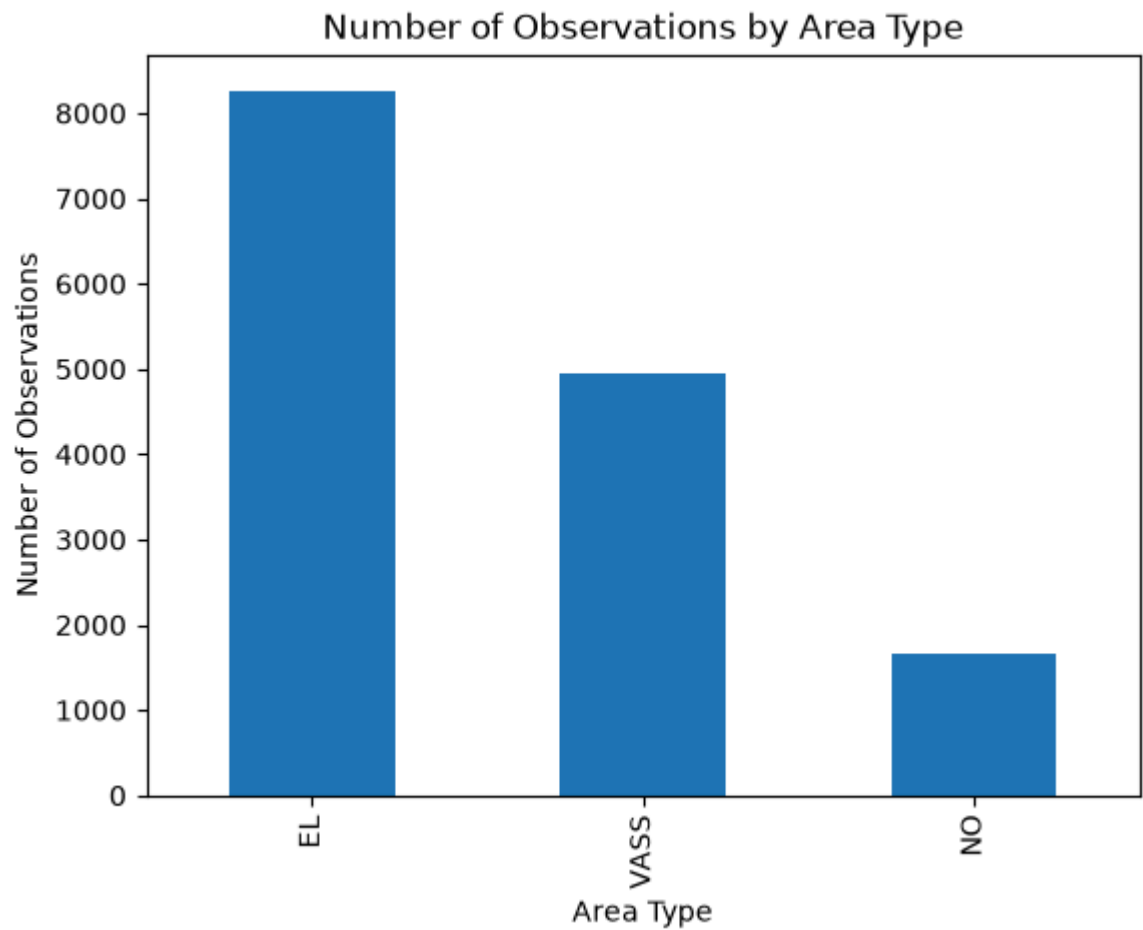
```

df["area_type"].value_counts().plot(kind="bar")

plt.title("Number of Observations by Area Type")
plt.xlabel("Area Type")
plt.ylabel("Number of Observations")

plt.show()

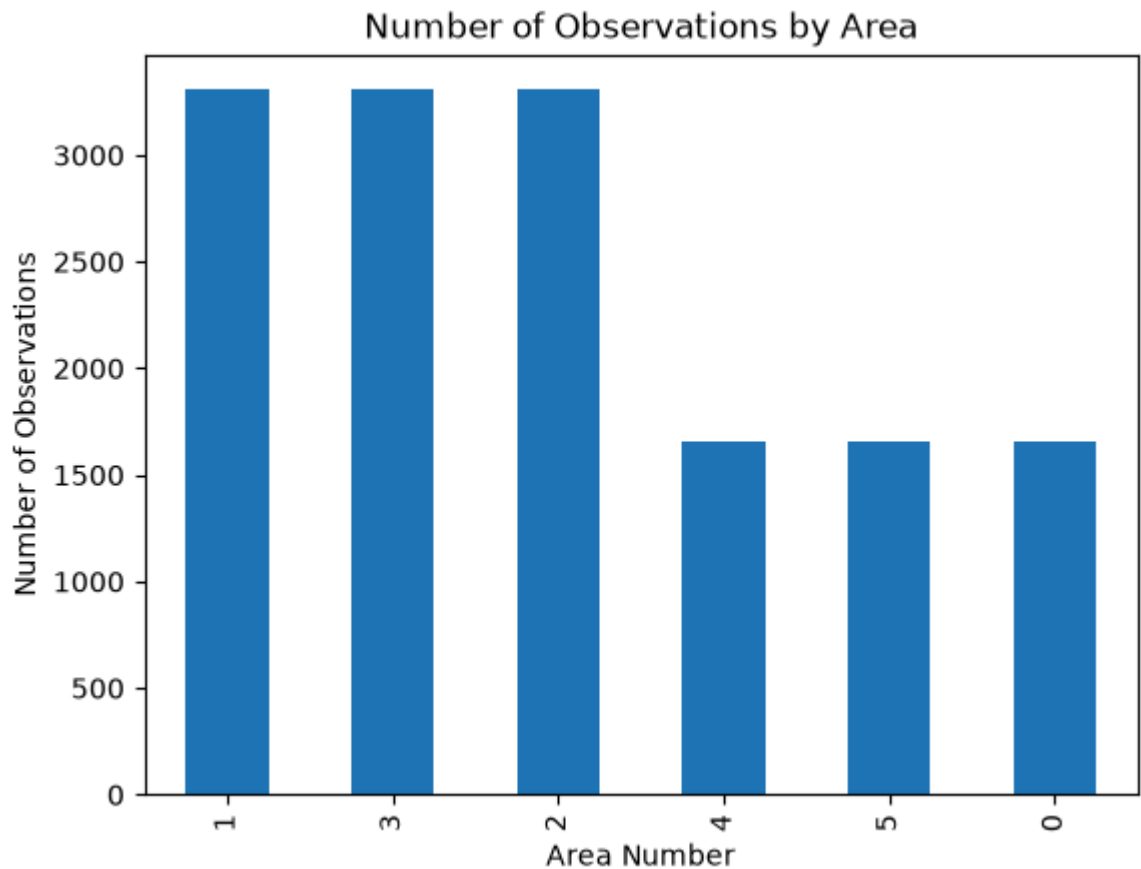
```



```
In [74]: # area_number: Identifies different geographical areas. A bar chart shows the nu
df["area_number"].value_counts().plot(kind="bar")

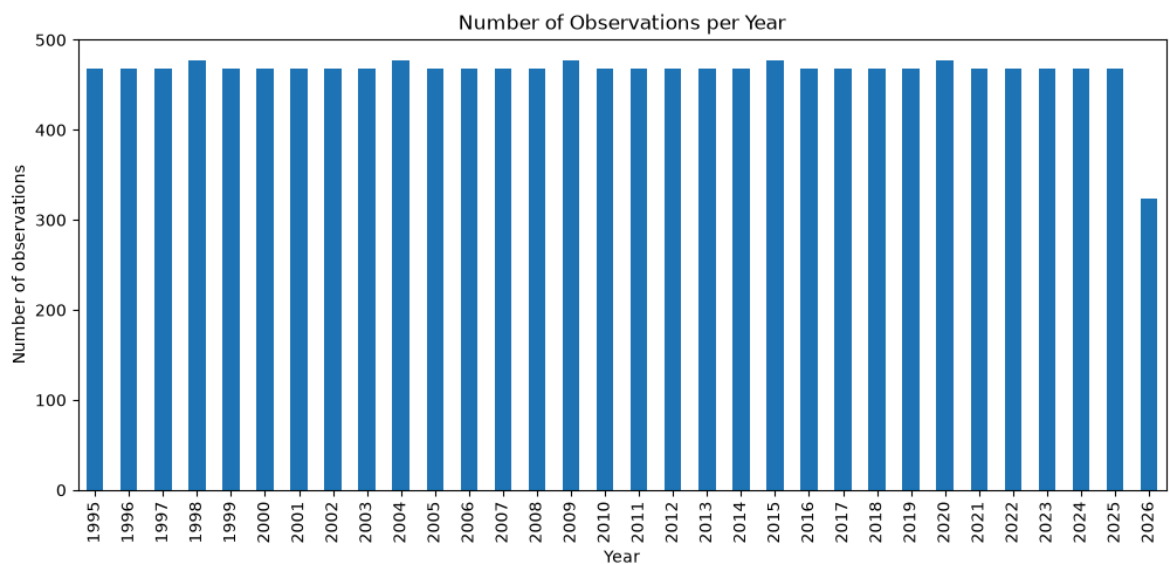
plt.title("Number of Observations by Area")
plt.xlabel("Area Number")
plt.ylabel("Number of Observations")

plt.show()
```



```
In [75]: #Number of observations sorted by year.
df["iso_year"].value_counts().sort_index().plot(kind="bar",  figsize=(12, 5))

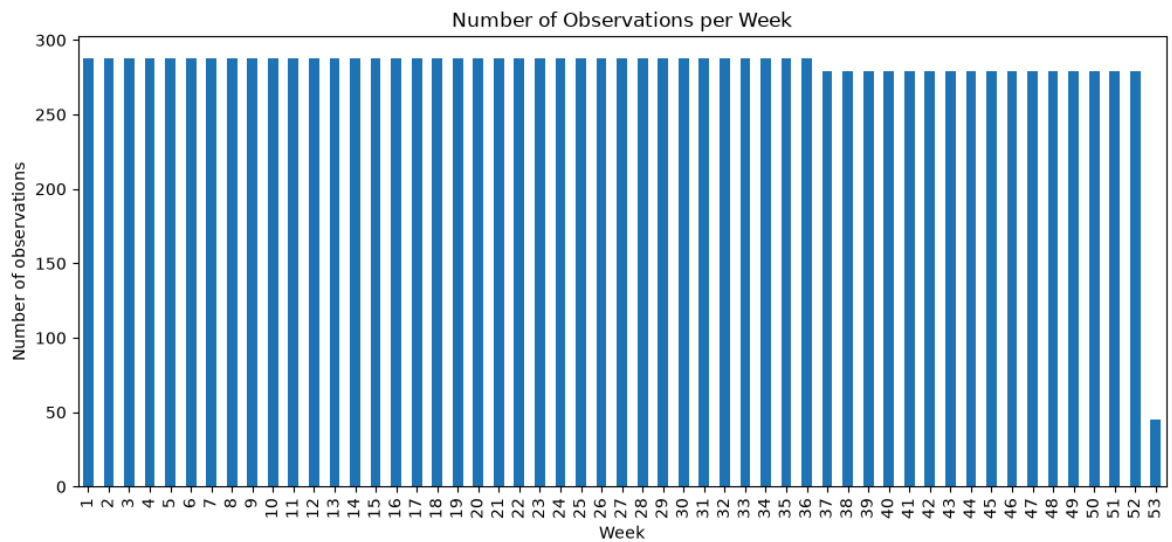
plt.xlabel("Year")
plt.ylabel("Number of observations")
plt.title("Number of Observations per Year")
plt.show()
```



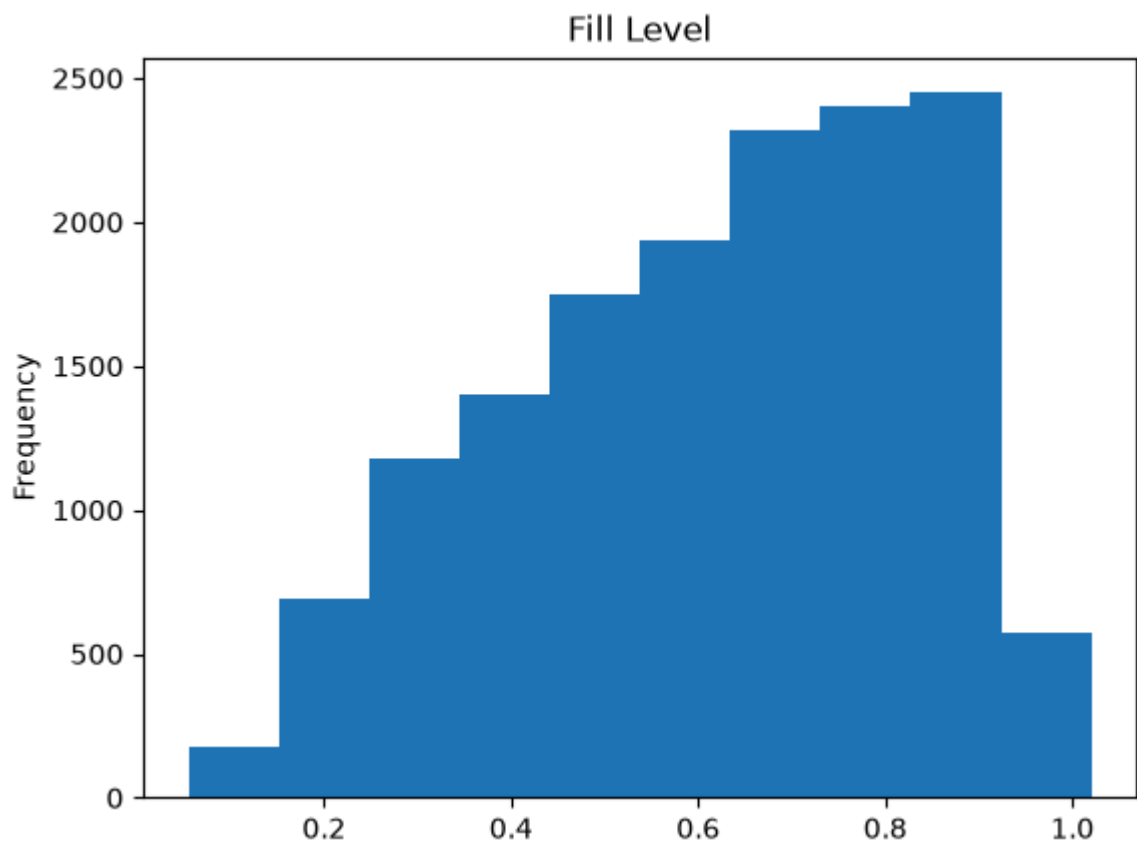
```
In [76]: #Number of observations per week
df["iso_week"].value_counts().sort_index().plot(kind="bar",  figsize=(12, 5))

plt.xlabel("Week")
plt.ylabel("Number of observations")
```

```
plt.title("Number of Observations per Week")
plt.show()
```

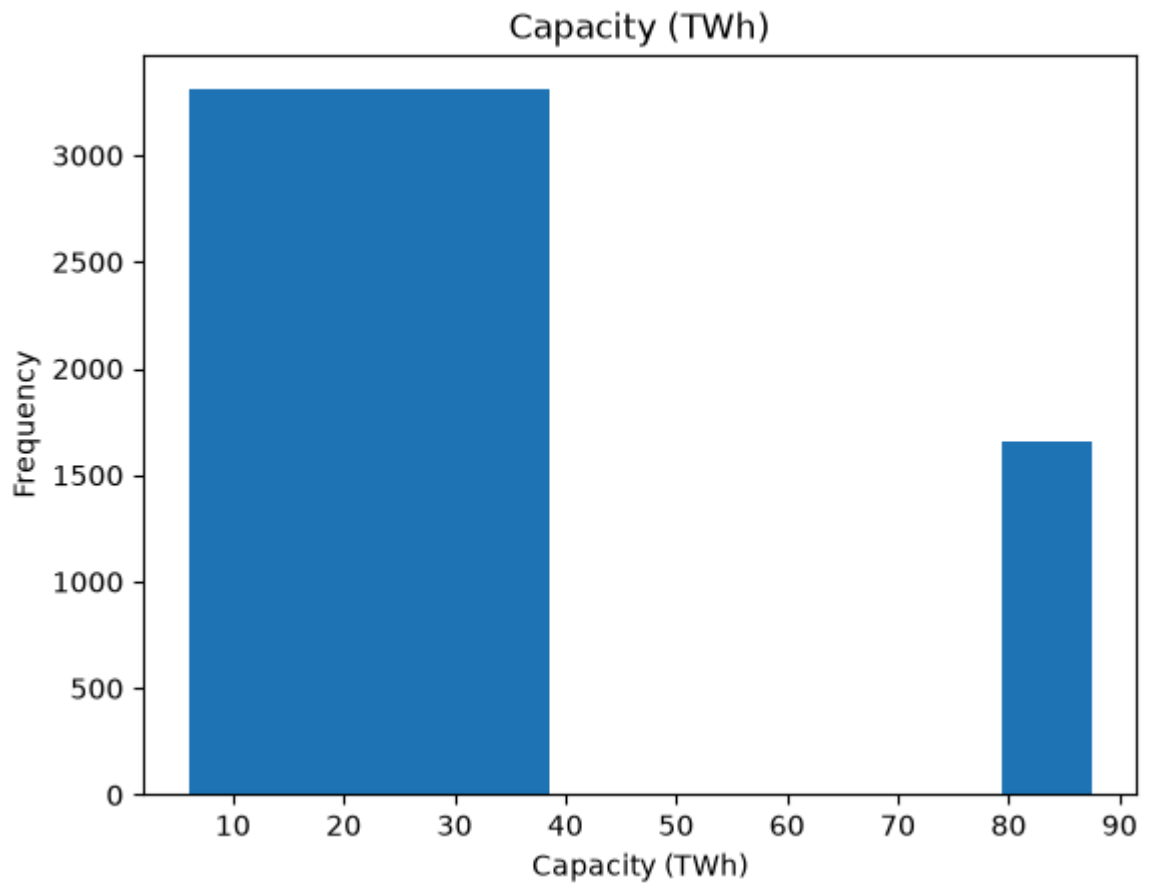


```
In [77]: # fill_level: Continuous numerical variable. A histogram shows the distribution
df["fill_level"].plot(kind="hist")
plt.title("Fill Level")
plt.show()
```

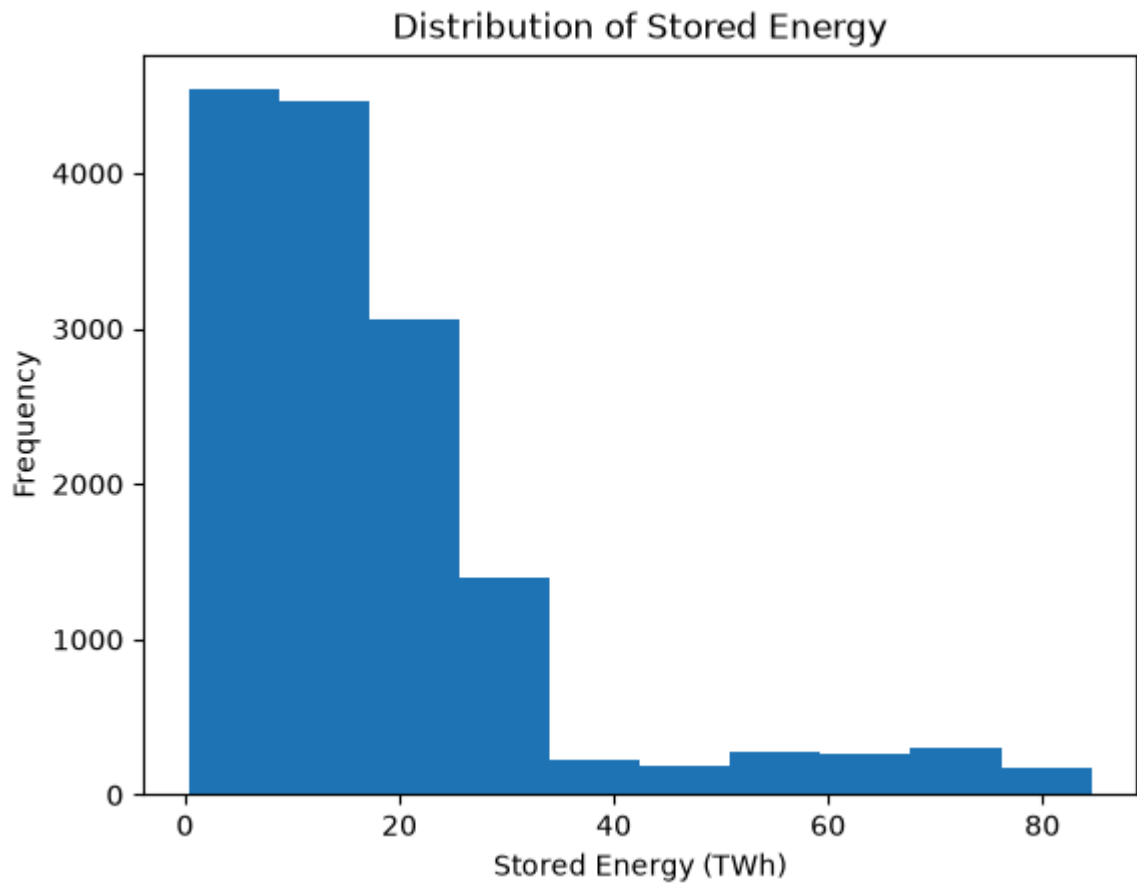


```
In [78]: # capacity_TWh: Continuous numerical variable. A histogram shows the distributio
df["capacity_TWh"].plot(kind="hist")
plt.title("Capacity (TWh)")
plt.xlabel("Capacity (TWh)")
plt.ylabel("Frequency")

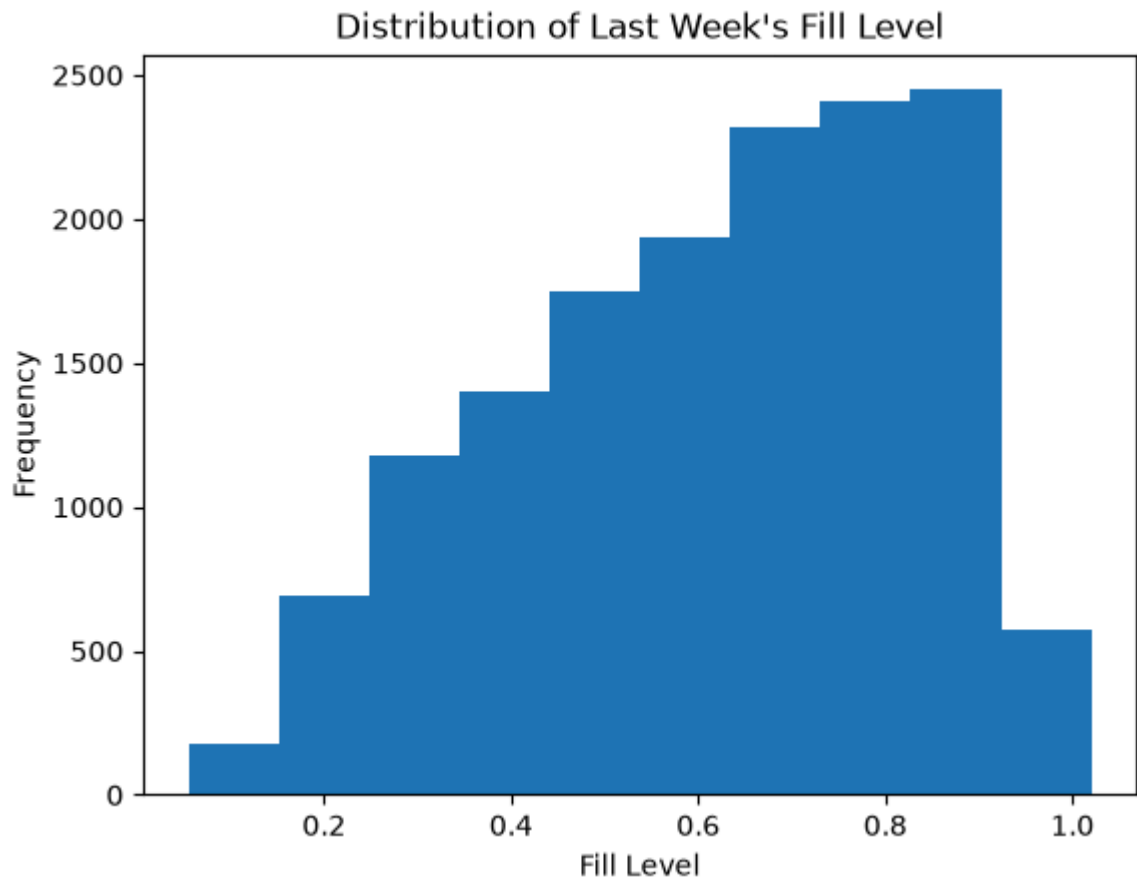
plt.show()
```



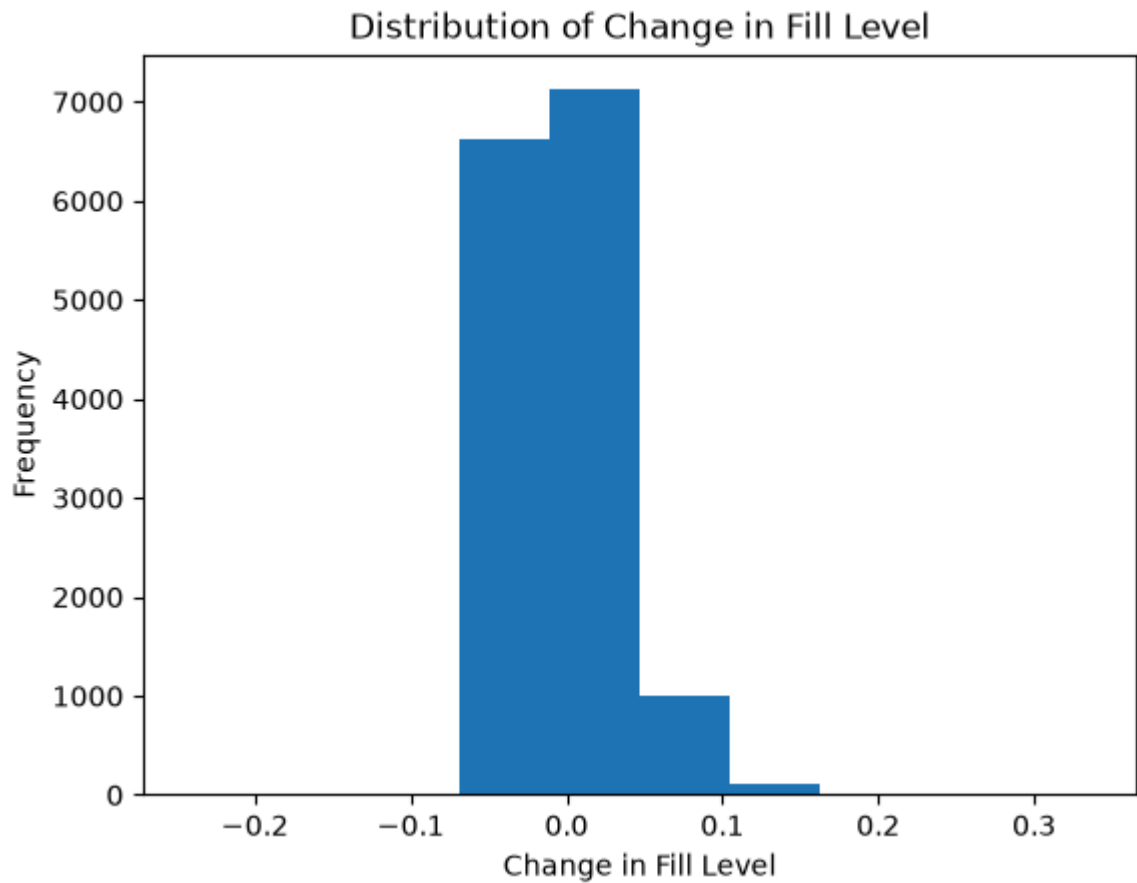
```
In [79]: # stored_energy_TWh: Continuous numerical variable.  
# A histogram shows the distribution of stored energy.  
  
df["stored_energy_TWh"].plot(kind="hist")  
  
plt.title("Distribution of Stored Energy")  
plt.xlabel("Stored Energy (TWh)")  
plt.ylabel("Frequency")  
  
plt.show()
```



```
In [80]: # fill_level_last_week: Continuous numerical variable.  
# A histogram shows the distribution of last week's fill levels.  
  
df["fill_level_last_week"].plot(kind="hist")  
  
plt.title("Distribution of Last Week's Fill Level")  
plt.xlabel("Fill Level")  
plt.ylabel("Frequency")  
  
plt.show()
```



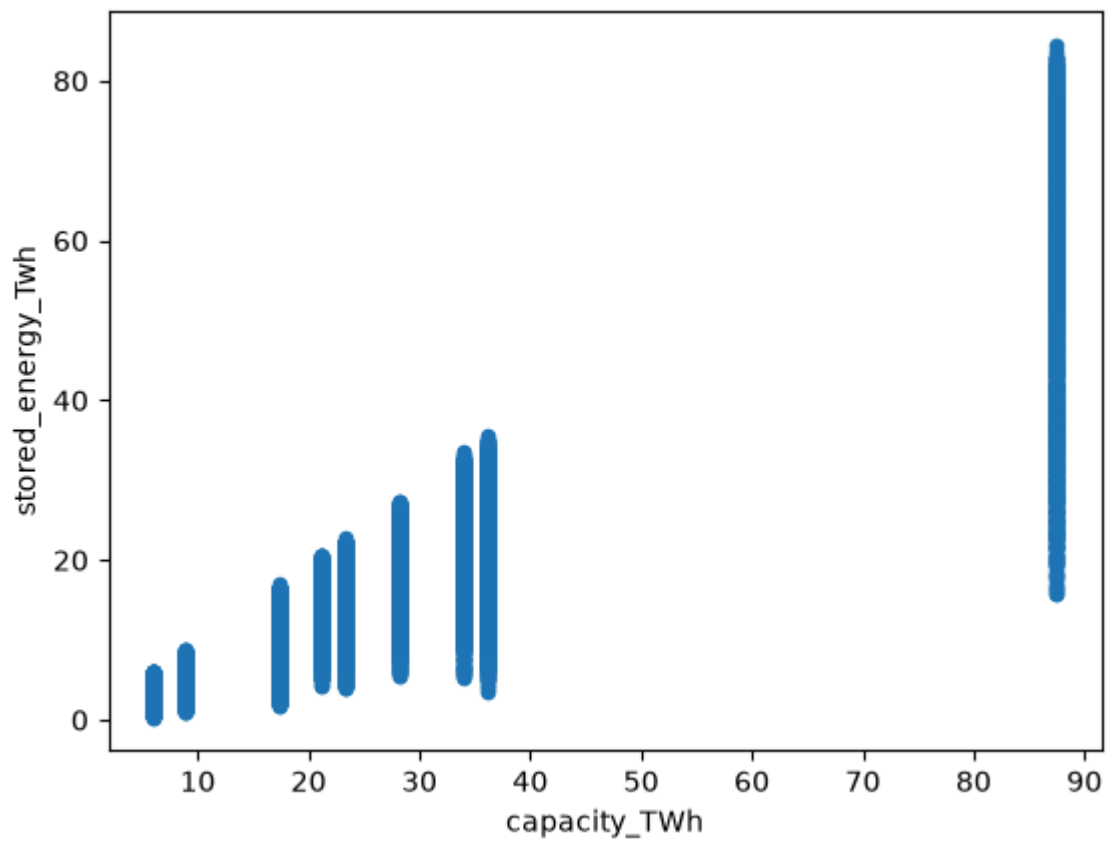
```
In [81]: # change_in_fill_level: Continuous numerical variable.  
# A histogram shows the distribution of changes in fill level.  
  
df["change_in_fill_level"].plot(kind="hist")  
  
plt.title("Distribution of Change in Fill Level")  
plt.xlabel("Change in Fill Level")  
plt.ylabel("Frequency")  
  
plt.show()
```



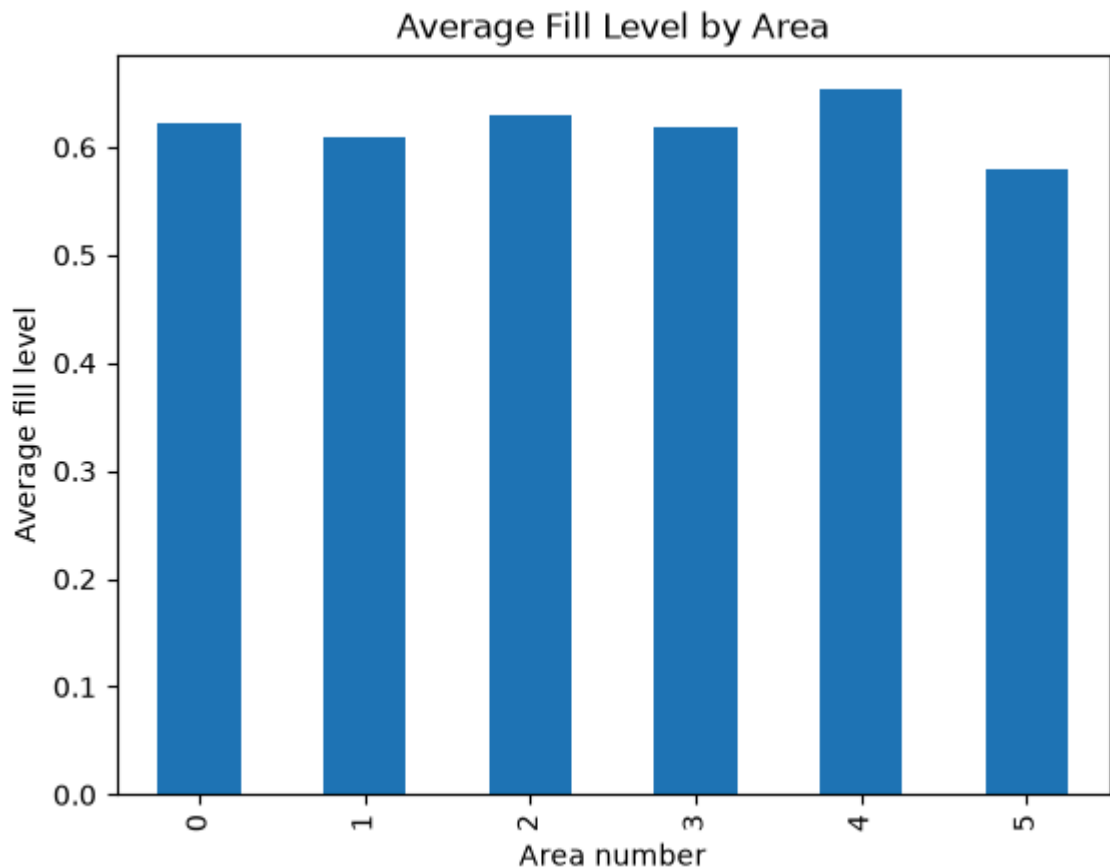
## Plotting Columns together

In [82]: *#Looking at the correlation between stored energy Twh and capacity Twh.*

```
df.plot(  
    x="capacity_Twh",  
    y="stored_energy_Twh",  
    kind="scatter"  
)  
  
plt.show()
```



```
In [83]: #Average fill level by area number.  
#Grouping the area number and filllevel. Plotting by the average fill level.  
  
df.groupby("area_number")["fill_level"].mean().plot(kind="bar")  
  
plt.xlabel("Area number")  
plt.ylabel("Average fill level")  
plt.title("Average Fill Level by Area")  
plt.show()
```



### All columns together

The measurement variables have different ranges, so `MinMaxScaler` was used to scale each variable between 0 and 1. This makes it possible to compare the patterns of the variables in the same plot. The dataset contains several reservoir areas for each date. Therefore, the data was grouped by date and the mean was calculated across the areas, giving one value for each variable per date. The plot shows that `fill_level` and `fill_level_last_week` follow very similar patterns. `stored_energy_TWh` also follows a repeating pattern, while `capacity_TWh` changes very little over time. `change_in_fill_level` fluctuates more because it represents the change in fill level rather than the fill level itself.

```
In [84]: from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# Measurement columns to compare
columns_to_scale = [
    "fill_level",
    "capacity_TWh",
    "stored_energy_TWh",
    "fill_level_last_week",
    "change_in_fill_level"
]

# Converting date to datetime
df["date_id"] = pd.to_datetime(df["date_id"])

# Creating a copy of the dataframe
df_scaled = df.copy()
```

```

# Scaling measurement columns between 0 and 1
scaler = MinMaxScaler()

df_scaled[columns_to_scale] = scaler.fit_transform(
    df[columns_to_scale]
)

# Calculating the mean across reservoir areas for each date
df_plot = (
    df_scaled
    .groupby("date_id")[columns_to_scale]
    .mean()
)

# Plotting all measurement columns together

df_plot.plot(
    figsize=(14, 7),
    style=["-", "-", "-", "--", "-"]
)

plt.title("Scaled Reservoir Data Over Time - Mean Across Areas")
plt.xlabel("Date")
plt.ylabel("Scaled Value (0-1)")
plt.legend(title="Variable")
plt.grid(alpha=0.3)
plt.tight_layout()

plt.show()

```

